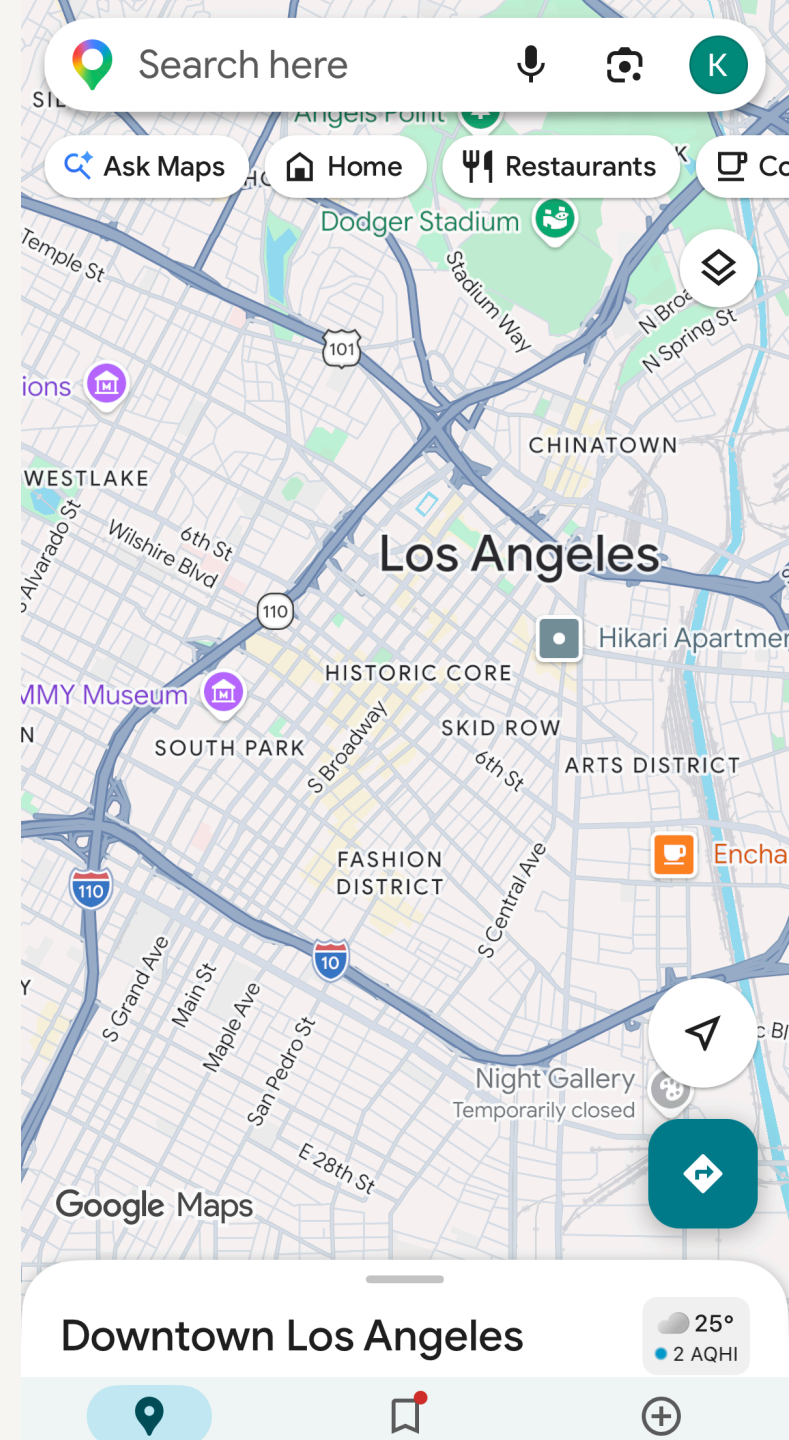


# Intro to Database and SQL



# What Is a Database?

A **structured** collection of data that's easy to **store, update, and search**.

Database =  
🧠 **organized data** + ⚙️ **software that manages it (DBMS)**

 1 of 11 

First Name

Urna

Last Name

Semper

Phone

(123) 456-7890

Email

no\_reply@example.com

Notes

# Why Do We Need Databases?

“Which customer bought Product A on January?”

**Without a database, you’d need to:**

1. Open and merge dozens of invoices manually.
2. Check for inconsistent column names and duplicate records.
3. Search through thousands of rows to find the right data.
4. Risk errors whenever someone updates an old file.

# Databases to the Rescue

- Store all sales records in **one structured place**.
- Query data instantly using SQL:

```
SELECT customer_name  
FROM sales  
WHERE product = 'A' AND month = 'January';
```

# Why Do We Need Databases?

- **Fast answers:**  
Query huge data sets in seconds instead of hours.
- **Keeps data clean and accurate:**  
Built-in rules prevent errors and duplicates.
- **Grows with you (Scalable):**  
Start small on a laptop → scale to millions of users.
- **Works for everyone (Shared access):**  
Multiple people or apps can use the same data safely.

# Two Key Concepts to Understand Databases

- **Entity Relationship Diagram (ERD)**
- **Structured Query Language (SQL)**

# Entity Relationship Diagram (ERD)

A **blueprint** of a database: how data is organized and connected

**Two elements:**

- **Entity (noun):** Something we store data about  
→ e.g., Student, Course, Enrollment
- **Relationship (verb):** How entities are linked  
→ e.g., Student "enrolls in" Course

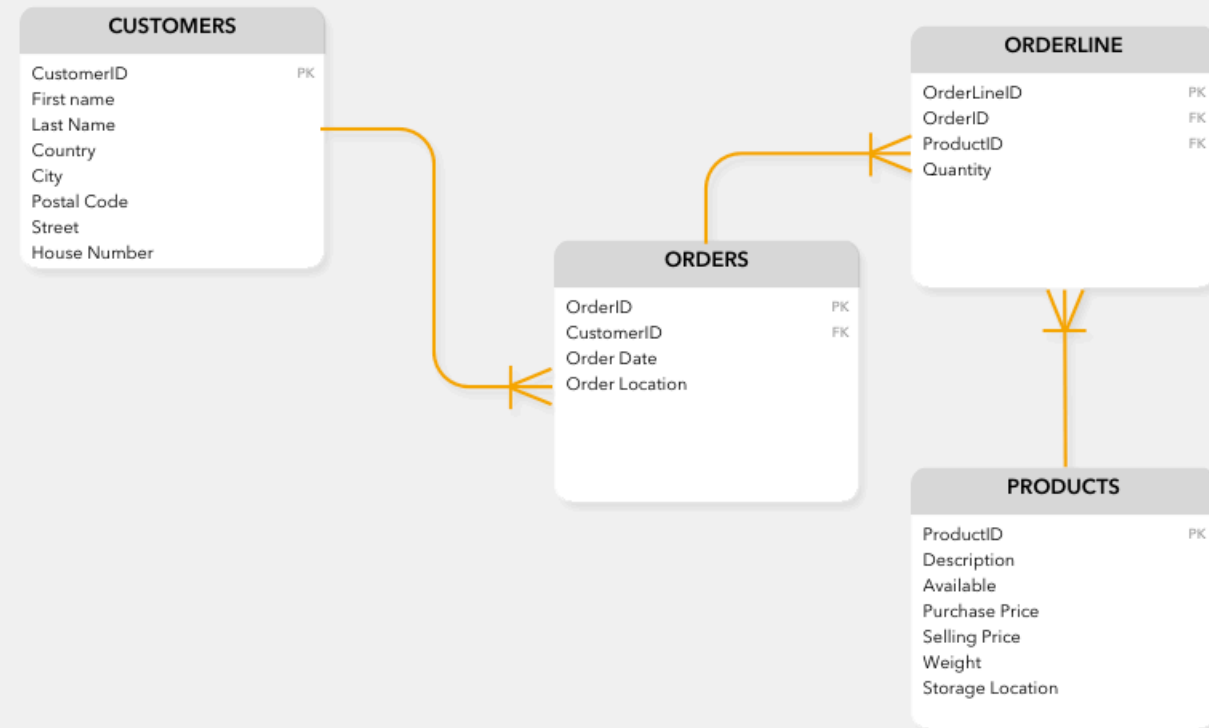


# Sample ERD

**Entities:** Customers, Orders, Products, Orderline

**Relationships:**

- Customers place Orders
- Orders contain Products via Orderline



# Structured Query Language (SQL)

To interact with databases



```
SELECT [ALL/DISTINCT] column_list
FROM table_list
[WHERE conditional expression]
[GROUP BY group_by_column_list]
[HAVING conditional expression]
[ORDER BY order_by_column_list]
```

Copyright ©2014 Pearson Education

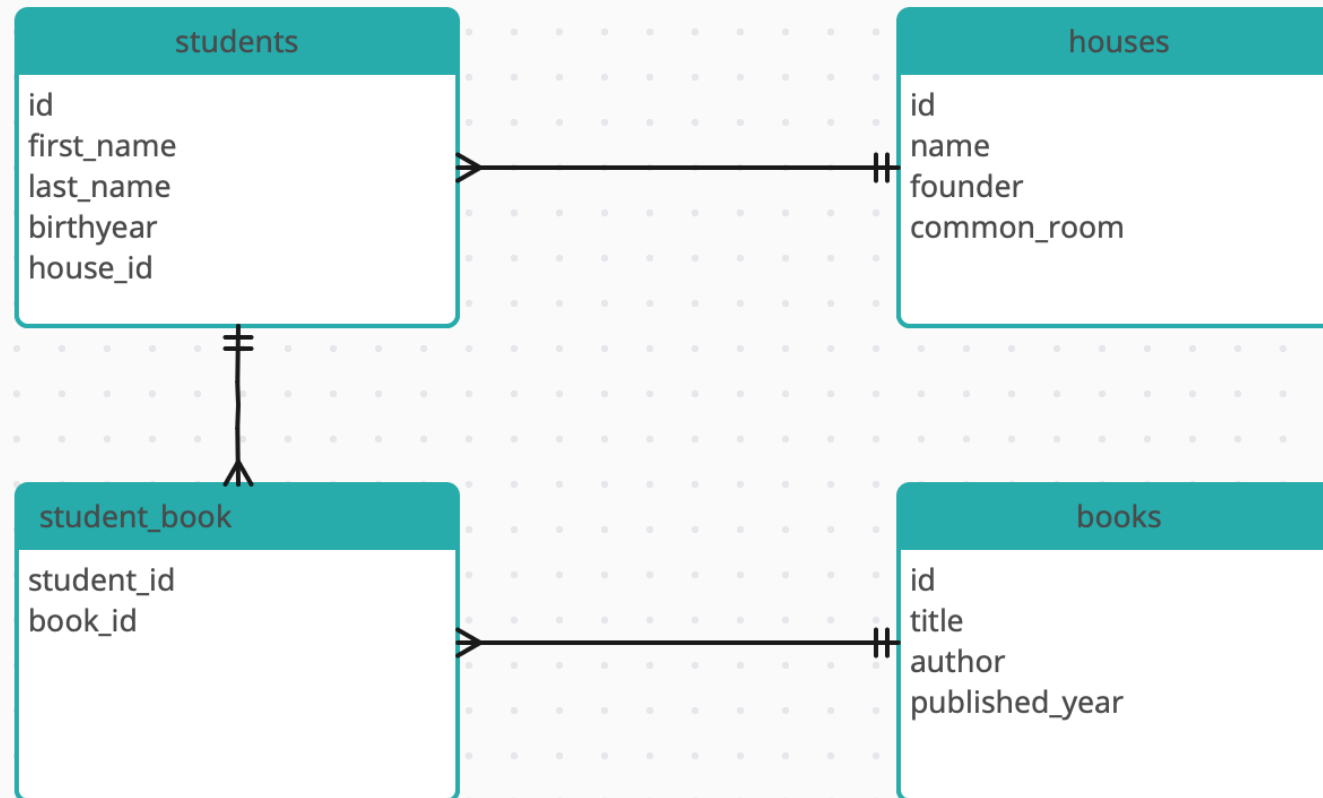
Essentials of Database Management, 1st ed.,  
Pearson.

# How to Design a Database

- 1 What do you want to store data about? (→Entities)
- 2 How are entities connected? (→Relationships)



# Harry Potter Database - ERD



# **SELECT** columns **FROM** a table

SELECT column\_name FROM table\_name

```
-- select first_name column from students table  
SELECT first_name FROM students;
```

```
-- select first_name and last_name columns from students table  
SELECT first_name, last_name FROM students;
```

```
-- select all columns from students table  
SELECT * FROM students;
```

**\*** is a wildcard that means "all columns"

**--** to add comments for your reference. Anything after **--** will be ignored by the database.

- *dash dash space* **--**, not *dash dash* **--**
- **CMD** + **/** to comment out a line on Mac, **CTRL** + **/** on Windows.

**;** to indicate the end of a SQL statement

Optional if you have only one statement.

**LIMIT** the number of records returned

```
SELECT column_name FROM table_name LIMIT number
```

```
SELECT * FROM students LIMIT 10;
```

**SELECT** columns **FROM** a table **WHERE** conditions  
are true

SELECT column\_name FROM table\_name WHERE condition

```
-- select first_name and last_name of all students in house_id = 1  
SELECT first_name, last_name FROM students WHERE house_id = 1;
```

```
-- select first_name and last_name of all students born in 1980  
SELECT first_name, last_name FROM students WHERE birthyear = 1980;
```

```
-- select first_name and last_name of all students born in or after 1980  
SELECT first_name, last_name FROM students WHERE birthyear >= 1980;
```





## Write SQL queries to answer the following questions

1. What year was Harry Potter born?
2. List all books that were published after 1999.
3. Who is the founder of Gryffindor?